We next establish a slight modification of IH-condition 5.6, namely that $w_q \in \mathit{to_be_explored}(E.q)$. The proof of this is contained in the proof of IH-condition 5.6: we first note in formula (12) that $p \notin I_{[E.q]}(w_q)$ for any $p \in \mathit{done}(E)$, then in formula (13) that $p \notin I_{[E.q]}(w_q)$ for any $p \in \mathit{init_sleep}(E)$, concluding that $I_{[E.q]}(w_q) \cap \mathit{init_sleep}(E.q) = \emptyset$.

We next claim that the algorithm will actually explore an execution sequence of form $E.q.w'$ which is in $[E.q.w_q.v'']_{\simeq}$ for some v'' (i.e., such that $w_q \sqsubseteq_{[E.q]} w'$), such that if w' contain some $\texttt{lock}(l)$ operation(s), then e_q is the first of these. To prove this claim, note that after $E.q.w_q$, process $\widehat{e_q}$ is prepared to perform a $\texttt{lock}(l)$ operation, and will continue to be prepared to do so all the time until it actually performs such a $\texttt{lock}(l)$ operation (note that immediately after $E.q.w_q$, the lock l is still taken). We split the proof into two cases:

(1) If $\mathit{to_be_explored}(E.q)$ contains a sequence of form $w_q.z$ in which z contains an $\texttt{unlock}(l)$ operation, then $\mathit{to_be_explored}(E.q)$ also contains such a sequence of form $w_q.z$ in which event e_q immediately follows the first of these $\texttt{unlock}(l)$ operations, since process $\widehat{e_q}$ is continuously prepared to perform $\texttt{lock}(l)$ operation after $E.q.w_q$. The inductive hypothesis for $E.q$ guarantees that the algorithm explores $E.q.w'$ for some w' with $z \sqsubseteq_{[E.q]} w'$. Since the operations $\texttt{lock}(l)$ are dependent with each other, e_q is the first event in w' to perform a $\texttt{lock}(l)$ operation.

(2) If $\mathit{to_be_explored}(E.q)$ contains no sequence of form $w_q.z$ in which z contains an $\texttt{unlock}(l)$ operation, then the inductive hypothesis for $E.q$ guarantees that the algorithm explores $E.q.w'$ for some w' with $w_q \sqsubseteq_{[E.q]} w'$. Since w' must be in $\mathit{to_be_explored}(E.q)$, we conclude that w' contains no $\texttt{unlock}(l)$, and hence no $\texttt{lock}(l)$ operation.

Having proven this claim, let u be the shortest prefix of w' such that $w_q \sqsubseteq_{[E.q]} u$. Note that after w_q, the event e_q (with operation $\texttt{lock}(l)$) is the next to be performed by $\widehat{e_q}$, although the lock l may still be taken immediately after u. Since, according to the just proven claim, u contains no $\texttt{lock}(l)$ operation, then $\mathit{next}_{[E]}(q)$ is the last event in $E.q.u$ that performs a $\texttt{lock}(l)$ operation, and so the race detection at lines 2–8 of Algorithm 3 will make sure that $\mathit{done}(E)$ contains some q' with $q' \in I_{[E]}(v)$, where $v = v'.\widehat{e_q}$ for $v' = \mathit{notdep}(\mathit{next}_{[E]}(q), E.q.u)$. In this case, we derive a contradiction in a similar way as in the proof of Claim 5.4. By the construction of v' we have $v' \sqsubseteq_{[E.q]} w'$. From $w_q \sqsubseteq_{[E.q]} w'$ and $v' \sqsubseteq_{[E.q]} w'$, and Property 4.7(3), we infer $w_q \sim_{[E.q]} v'$. This implies, using $E \vdash q \diamond w_q$ and $E \vdash q \diamond v'$, that $w_q \sim_{[E]} v'$ (this can be established in a similar way as Lemma 5.1.3). Now $q' \in I_{[E]}(v)$ means that either (i) $q' \in I_{[E]}(v')$, or that (ii) $q' = \widehat{e_q}$ and $E \vdash q' \diamond v'$. Case (i) implies (by the construction of v') that $\mathit{next}_{[E]}(q')$ does not perform any operation on the lock l, and (using $w_q \sim_{[E]} v'$) that $q' \in \mathit{WI}_{[E]}(w_q)$. Since e_q performs the operation $\texttt{lock}(l)$, this implies $q' \in \mathit{WI}_{[E]}(w_R)$, which contradicts Property (10). In Case (ii), from $w_q \sqsubseteq_{[E.q]} u$ and $E \vdash q \diamond w_q$, the construction of v' implies $w_q \sqsubseteq_{[E.q]} v'$. Also considering $E \vdash q \diamond v'$ we infer $w_q \sqsubseteq_{[E]} v'$. Since $q' \in \mathit{done}(E)$, the construction of w_R implies that $E \nvdash q' \diamond w_q$, which together with $w_q \sqsubseteq_{[E]} v'$ implies $E \nvdash q' \diamond v'$, contradicting the assumption $E \vdash q' \diamond v'$ for case (ii).

This concludes the proof of Claim 5.4. The rest of the proof proceeds as in the proof of Theorem 5.2. □

8.1.3 Extending Optimal-DPOR to Handle Locks. Let us next present our extension of Algorithm 2 to programs with locks, resulting in Algorithm 4. Similarly as in Algorithm 3, the algorithm extends the lock detection phase to consider races between $\texttt{lock()}$ operations, one of which is potentially disabled. The extension for locks is handled by lines 8–14. These lines handle the case when event e' in a race $e \precsim_E e'$ is actually blocked for the reason that both e and e' are $\texttt{lock}(l)$ operations on the same lock l. In this case, e' may not be present in the execution sequence E at a position where it is blocked. For this case, we construct the sequence v as in line 6 of Algorithm 3. First, at line 10,